1. Identificación y contexto

Chatbot del Hotel Tequendama

Asistente conversacional para reservar servicios del hotel

- Integrantes: Andres David Hernandez y Juan Jose Usma.
- Curso: Electiva II (EAM-2026II) · Entrega del segundo corte.
- El problema: el hotel recibe solicitudes de reserva de masajes, spa, cenas, salas de reuniones y tours, y debe evitar que dos reservas se crucen en el mismo horario.
- La idea: que el huésped escriba en lenguaje natural (por ejemplo, 'quiero un masaje mañana en la tarde') y el sistema consulte, valide y guarde la reserva por él.

Repositorio: [COMPLETAR enlace de GitHub]

2. Solución y alcance

Qué hace

Conversa en español con memoria · Consulta servicios y precios · Muestra horarios libres · Crea, consulta, modifica y cancela reservas · Pide confirmación antes de guardar cualquier cambio

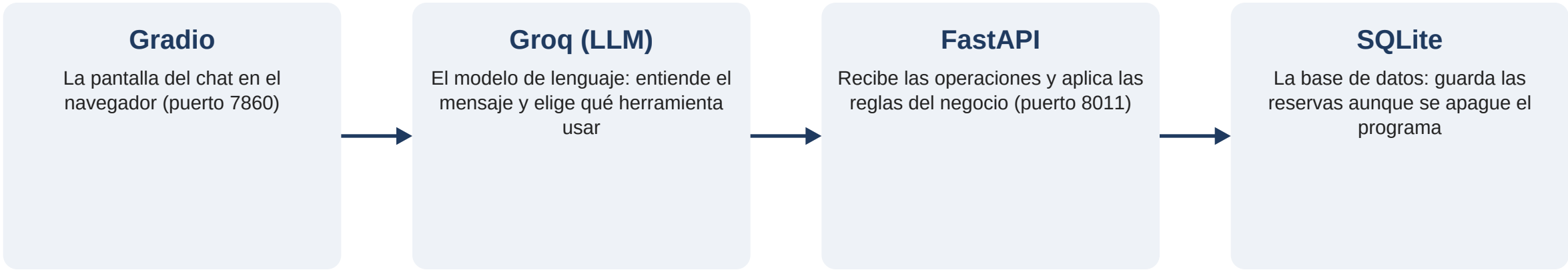
Qué no hace (aún)

No reserva noches de habitación (solo las muestra como información) · No tiene usuarios ni pagos · Usa una sola agenda para todo el hotel · No está publicado en internet

Datos de prueba

Todo el catálogo es ficticio: 3 tipos de habitación y 5 servicios (masaje 60 min, spa 90, cena romántica 120, sala de reuniones 120, tour 180) con su precio en pesos colombianos.

3. Funcionamiento: cuatro piezas



Cómo se conectan

- El modelo no toca la base de datos: solo pide 'herramientas' (listar servicios, ver disponibilidad, crear, modificar, cancelar).
- El código del chat ejecuta esas herramientas llamando a la API por HTTP; la API es la única que escribe en SQLite.
- Las escrituras (crear, modificar, cancelar) esperan a que el usuario pulse 'Confirmar operación'.

4. Recorrido de una solicitud completa

1	**El usuario escribe**	'Quiero un spa mañana a las 3 pm. Soy Ana Gómez.'
2	**Groq consulta**	Pide la disponibilidad real del spa; la API responde con las horas libres.
3	**Resumen**	El chat muestra cliente, servicio, fecha y hora, y se detiene a esperar.
4	**Confirmar**	El usuario pulsa 'Confirmar operación'; recién ahí se llama a la API.
5	**Guardado**	La API valida el horario y guarda en SQLite; responde con un código de reserva.

Si el usuario pulsa 'Descartar solicitud', no se guarda nada.

5. Reglas del negocio

- Horario: todos los días de 08:00 a 20:00, con horas en pasos de 30 minutos.
- Duración: cada servicio dura lo indicado en el catálogo y debe terminar antes del cierre (una cena de 2 horas no puede empezar a las 19:00).
- Sin fechas pasadas.
- Sin cruces: hay una sola agenda, así que dos reservas activas no pueden solaparse, aunque sean de servicios distintos. Una que empieza justo cuando otra termina sí es válida.
- Cancelar no borra: la reserva queda como 'cancelada', se conserva el historial y el horario se libera.
- Comprobar y guardar ocurren en una sola transacción, así dos solicitudes simultáneas no pueden tomar el mismo horario.

6. Pruebas y resultados

Prueba	Resultado	Estado
Cruce exacto y parcial de horarios	Rechazado (409)	Aprobada
Reserva pegada a otra (11:00 tras 10:00-11:00)	Permitida	Aprobada
Termina después del cierre / hora fuera de paso	Rechazado (422)	Aprobada
Fecha pasada, servicio o nombre inválido	Rechazado (422)	Aprobada
Modificar, cancelar y reutilizar el horario liberado	Funciona	Aprobada
Cancelar dos veces / modificar una cancelada	Rechazado (409)	Aprobada
Reiniciar la agenda: las filas se conservan	3 filas antes y después	Aprobada

16 pruebas de la capa de reservas aprobadas. Pendientes de documentar con capturas: conversación de varios turnos, dos sesiones, confirmar/descartar y reinicio de la API. [COMPLETAR resultados]

7. Evolución frente a Flowise + JSON

Aspecto	Corte 1 (Flowise)	Corte 2 (Gradio + FastAPI + SQLite)
Control del flujo	Nodos visuales	Código propio, cada paso se ve y se prueba
Persistencia	Archivo JSON	SQLite con transacciones
Disponibilidad	Marcas disponible / no	Intervalos con duración y detección de cruces
Escrituras seguras	Dependía del prompt	Confirmación en código + validación en la API
Qué se reserva	Noches de habitación	Servicios por franja horaria

Pasos del camino: Flowise y JSON (12 sep) → Gradio y Groq con herramientas (19 oct) → SQLite y reglas de disponibilidad.

8. Cierre

Limitaciones y mejoras pendientes

- Reservar noches de habitación (check-in y check-out).
- Una agenda por servicio o con capacidad, en lugar de una sola para todo el hotel.
- Autenticación de usuarios y pruebas automáticas del flujo con el modelo.
- Guardar el historial de la conversación para que no se pierda al reiniciar el chat.

Qué se aprendió

- Una regla crítica no debe depender solo del prompt: se impone en el código.
- Comprobar y guardar en la misma transacción evita reservas duplicadas.

Código, README y bitácora: [COMPLETAR enlace de GitHub]